Aplikacja webowa do przeglądania filmów z integracją TMDB API oraz wsparciem AI

Autor: Artem Ostapchuk, 511120

Kierunek: Technologii Komputerowe

Prowadzący: dr. Wojciech Czart

Data: 16 lutego 2026 r.

1. Streszczenie projektu

Celem projektu było stworzenie kompletnej aplikacji webowej opartej na technologii **ASP.NET Core MVC**, umożliwiającej przeglądanie filmów, filtrowanie ich według gatunków i lat, wyświetlanie szczegółowych informacji o każdym tytule oraz automatyczny import danych z zewnętrznego API — The Movie Database (TMDB). Projekt został zaprojektowany jako nowoczesna aplikacja internetowa, która łączy klasyczne techniki programistyczne z zaawansowanymi narzędziami sztucznej inteligencji.

Aplikacja pozwala użytkownikowi na intuicyjne przeglądanie bazy filmów, korzystając z przejrzystego interfejsu oraz funkcjonalnych filtrów. Każdy film posiada własną stronę szczegółów, zawierającą opis fabuły, obsadę, reżysera, ocenę IMDb oraz plakat. Dane te są pobierane automatycznie z TMDB, co eliminuje konieczność ręcznego wprowadzania informacji i zapewnia ich aktualność.

Projekt został wzbogacony o intensywne wykorzystanie narzędzi sztucznej inteligencji, które wspierały cały proces tworzenia aplikacji. AI pomagało w:

- generowaniu kodu C# i Razor,
- analizie błędów i ich szybkim rozwiązywaniu,
- projektowaniu interfejsu użytkownika,
- optymalizacji logiki importu danych,
- tworzeniu mechanizmu lokalizacji (język polski + fallback angielski),
- usprawnianiu struktury projektu i organizacji repozytorium.

Dzięki temu możliwe było znaczne przyspieszenie pracy oraz uniknięcie wielu typowych problemów związanych z integracją API i budową aplikacji webowych.

Ważnym elementem projektu było również zapewnienie poprawnej lokalizacji danych filmowych. TMDB nie zawsze udostępnia polskie tłumaczenia opisów, dlatego w aplikacji zaimplementowano mechanizm automatycznego przełączania na opis angielski w przypadku braku wersji polskiej. Pozwoliło to zachować spójność i kompletność danych prezentowanych użytkownikowi.

Efektem końcowym jest w pełni funkcjonalna aplikacja przypominająca uproszczoną wersję serwisów takich jak Filmweb czy Netflix. Umożliwia ona wygodne przeglądanie filmów, szybkie filtrowanie treści oraz dostęp do szczegółowych informacji o każdym tytule. Projekt stanowi praktyczne połączenie technologii webowych, integracji API oraz narzędzi sztucznej inteligencji, a także pokazuje, jak AI może realnie wspierać proces tworzenia oprogramowania.

2. Wprowadzenie

Projekt został wykonany jako zadanie semestralne, którego głównym celem było połączenie klasycznego programowania z nowoczesnymi narzędziami sztucznej inteligencji. Współczesne aplikacje webowe coraz częściej wykorzystują zewnętrzne API, automatyzację oraz inteligentne mechanizmy wspierające proces tworzenia oprogramowania. Dlatego zdecydowałem się na realizację projektu, który łączy wszystkie te elementy w jednym, spójnym rozwiązaniu.

Temat aplikacji filmowej został wybrany świadomie, ponieważ:

- **pozwala na pracę z realnym API** (The Movie Database – TMDB), co daje możliwość pobierania aktualnych danych filmowych i ich dynamicznej prezentacji,
- **wymaga stworzenia bazy danych**, dzięki czemu mogłem przećwiczyć pracę z Entity Framework Core oraz utrwalaniem danych lokalnie,
- **wymaga zaprojektowania interfejsu użytkownika**, co pozwoliło mi rozwinąć umiejętności związane z HTML, CSS i Razor,
- **umożliwia wykorzystanie AI do automatyzacji wielu elementów**, takich jak generowanie kodu, analiza błędów czy projektowanie widoków,
- **jest praktyczny i atrakcyjny wizualnie**, co sprawia, że efekt końcowy jest nie tylko funkcjonalny, ale również przyjemny w odbiorze.

W trakcie realizacji projektu pojawiło się kilka wyzwań technicznych, które wymagały zastosowania bardziej zaawansowanych rozwiązań. Jednym z nich była konieczność zapewnienia **lokalizacji polskiej**. TMDB nie zawsze udostępnia polskie tłumaczenia opisów filmów, dlatego konieczne było zaimplementowanie mechanizmu fallbacku językowego — w sytuacji, gdy opis w języku polskim nie jest dostępny, aplikacja automatycznie pobiera wersję angielską. Dzięki temu dane prezentowane użytkownikowi są zawsze kompletne i spójne.

Projekt pozwolił mi zdobyć praktyczne doświadczenie w wielu obszarach:

- pracy z frameworkiem **ASP.NET Core MVC**,
- integracji z zewnętrznymi API i przetwarzaniu danych JSON,
- projektowaniu i zarządzaniu **bazą danych SQLite**,
- tworzeniu responsywnego interfejsu użytkownika,
- wykorzystywaniu **narzędzi AI** jako wsparcia w procesie programowania,
- analizie błędów i optymalizacji kodu.

Realizacja projektu pokazała mi również, jak duże znaczenie ma odpowiednia organizacja kodu, struktura repozytorium oraz umiejętność łączenia różnych technologii w jednym rozwiązaniu. Dzięki temu projektowi mogłem nie tylko poszerzyć swoją wiedzę, ale także zrozumieć, jak wygląda proces tworzenia nowoczesnych aplikacji webowych w praktyce.

3. Technologie użyte w projekcie

Projekt został zrealizowany z wykorzystaniem kilku kluczowych technologii, które umożliwiły stworzenie funkcjonalnej, stabilnej i nowoczesnej aplikacji webowej. Każda z nich pełniła określoną rolę i wspierała inny aspekt działania systemu.

3.1 ASP.NET Core MVC

Głównym frameworkiem użytym w projekcie był **ASP.NET Core MVC**, który zapewnia architekturę Model–View–Controller. Dzięki temu możliwe było:

- oddzielenie logiki od warstwy prezentacji,
- łatwe zarządzanie routingiem,
- szybkie tworzenie widoków Razor,
- utrzymanie przejrzystej struktury projektu.

Framework ten jest wydajny, nowoczesny i powszechnie stosowany w aplikacjach webowych.

3.2 Entity Framework Core + SQLite

Do przechowywania danych wykorzystano **Entity Framework Core** oraz lekką bazę danych **SQLite**. EF Core umożliwił:

- pracę z bazą danych w sposób obiektowy,
- automatyczne tworzenie tabel,
- łatwe zapisywanie i odczytywanie danych.

SQLite jest szybka, prosta w konfiguracji i idealna do projektów semestralnych.

3.3 TMDB API

Aplikacja integruje się z **The Movie Database (TMDB)**, pobierając:

- listę popularnych filmów,
- szczegóły,
- obsadę,
- gatunki,
- plakaty.

Zastosowano **lokalizację PL** oraz **fallback EN**, aby zapewnić kompletność danych.

3.4 CSS i interfejs użytkownika

Interfejs został zaprojektowany z użyciem własnych stylów CSS. Zastosowano:

- czerwone przyciski filtrów,
- responsywną siatkę filmów,
- estetyczne karty filmów.

Dzięki temu aplikacja jest przejrzysta i przyjazna dla użytkownika.

4. Aspekt AI w projekcie

Sztuczna inteligencja odegrała istotną rolę w realizacji projektu, wspierając zarówno proces programowania, jak i analizę problemów technicznych. AI była wykorzystywana jako narzędzie pomocnicze, które przyspieszało pracę, podpowiadało rozwiązania oraz pomagało w tworzeniu bardziej przejrzystego i zoptymalizowanego kodu.

4.1 Wykorzystane platformy i modele AI

W projekcie korzystałem z następujących narzędzi opartych na sztucznej inteligencji:

- **Microsoft Copilot (model GPT-4 / GPT-4o)** — generowanie kodu, analiza błędów, pomoc w strukturze kontrolerów, modeli i widoków.
- **ChatGPT (model GPT-4o)** — wsparcie w pisaniu dokumentacji, tłumaczeniach, wyjaśnieniach technicznych oraz poprawie jakości opisów.

4.2 Wsparcie w generowaniu kodu

AI pomagała w tworzeniu:

- metod kontrolerów,
- logiki filtrowania filmów,
- struktury widoków Razor,
- fragmentów CSS,
- kodu odpowiedzialnego za integrację z TMDB API.

Dzięki temu możliwe było szybkie budowanie funkcjonalności bez konieczności pisania wszystkiego od podstaw.

4.3 Analiza błędów

AI wspierała proces debugowania, wskazując:

- błędne ścieżki routingu,
- problemy z deserializacją JSON,

- brakujące dane w odpowiedziach API,
- błędy wynikające z duplikatów w bazie danych.

Pozwoliło to szybciej rozwiązywać problemy i utrzymać stabilność aplikacji.

4.4 Projektowanie interfejsu

AI pomogła również w tworzeniu:

- nowoczesnych przycisków filtrów,
- responsywnego układu kart filmów,
- estetycznych stylów CSS.

Dzięki temu interfejs aplikacji jest spójny i przyjazny dla użytkownika.

4.5 Przykłady promptów

- *„Napisz importer TMDB w C#, który pobiera filmy i obsadę oraz zapisuje je do bazy SQLite."*
- *„Stwórz czerwone przyciski filtrów w stylu Netflixa."*
- *„Dlaczego widok ByGenre zwraca błąd 404? Podpowiedz rozwiązanie."*

4.6 Ograniczenia AI

Mimo dużej pomocy, AI:

- czasem generowała błędne ścieżki,
- nie widziała rzeczywistych danych z TMDB,
- wymagała weryfikacji wygenerowanego kodu.

Dlatego AI była wsparciem, ale nie zastępowała samodzielnego myślenia

4.7 Zrzuty ekranu

„Błąd *no such column: m.Rating* oznacza, że w Twojej bazie SQLite **nigdy nie została utworzona kolumna** `Rating`, mimo że dodałeś ją w modelu `Movie`. To bardzo typowa sytuacja po dodaniu nowego pola do modelu w Entity Framework Core.

Dlaczego tak się stało?

Dodałeś w modelu:

Csharp Копіювати

```
public double Rating { get; set; }
```

Ale baza danych **nie została zaktualizowana**, ponieważ SQLite nie tworzy nowych kolumn automatycznie. Jeśli nie wykonałeś migracji, baza nadal działa na starej strukturze — bez kolumny `Rating`.

Jak to naprawić?

1. Utwórz nową migrację:

Rysunek 5. "Analiza błędu kolumny Rating"

Jak rozwiązać problem

1. **Zmiana nazwy widoku**
   W folderze *Views/Movie* powinien znajdować się plik:

   Код Копіювати

   ```
   ByGenre.cshtml
   ```

   Jeśli nazywał się „ByGenreView.cshtml", trzeba zmienić go na poprawną nazwę.
2. **Poprawienie wywołania widoku w kontrolerze**
   W metodzie kontrolera powinno być:

   Csharp Копіювати

   ```
   return View("ByGenre", movies);
   ```

   a nie:

   Csharp Копіювати

   ```
   return View("ByGenreView", movies);
   ```

3. Sprawdzenie routingu

Rysunek 6. "Poprawa widoku ByGenre"

⭐ KROK 2 — Dodajemy filtr po roku

🔧 1. Dodaj metodę w `MoviesController`

Csharp Копіювати

```csharp
public IActionResult ByYear(int year)
{
    var movies = _context.Movies
        .Where(m => m.Year == year)
        .ToList();

    ViewBag.Year = year;

    return View(movies);
}
```

🔧 2. Dodaj zakładkę „Lata" w menu

W `_Layout.cshtml` dodaj:

Rysunek 7. "Dodajemy filtr po roku"

5. Opis działania aplikacji

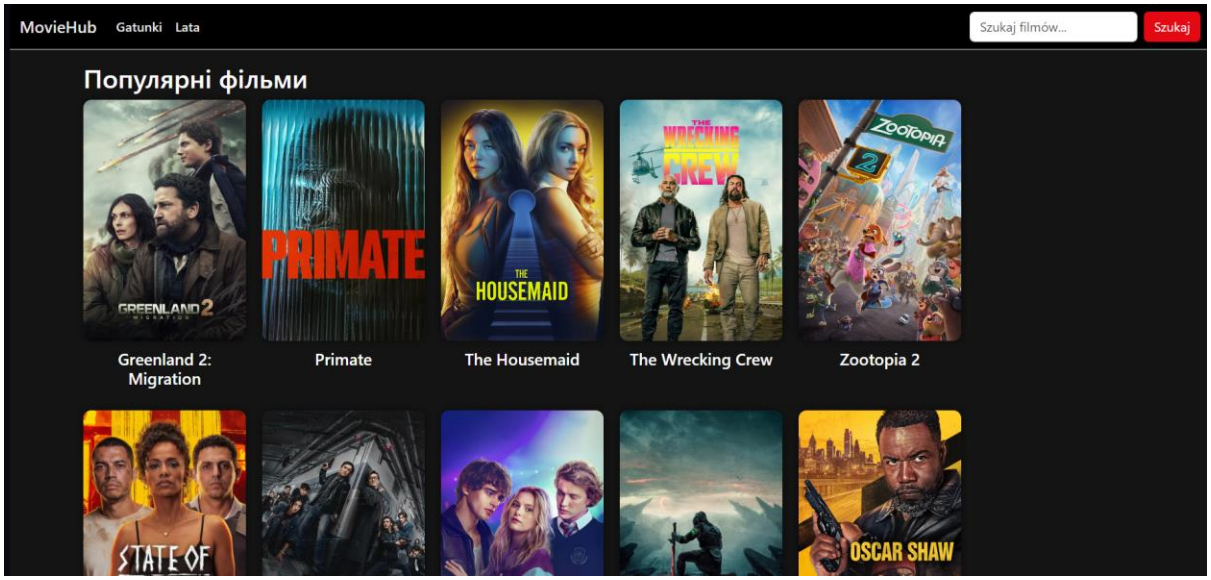
Aplikacja została zaprojektowana jako proste i intuicyjne narzędzie do przeglądania filmów. Składa się z kilku widoków, które umożliwiają użytkownikowi szybkie odnajdywanie interesujących tytułów oraz dostęp do szczegółowych informacji.

5.1 Strona główna

Strona główna wyświetla listę filmów pobranych z TMDB i zapisanych w lokalnej bazie danych. Każdy film jest prezentowany jako karta zawierająca:

- plakat,
- tytuł,
- rok produkcji,
- ocenę.

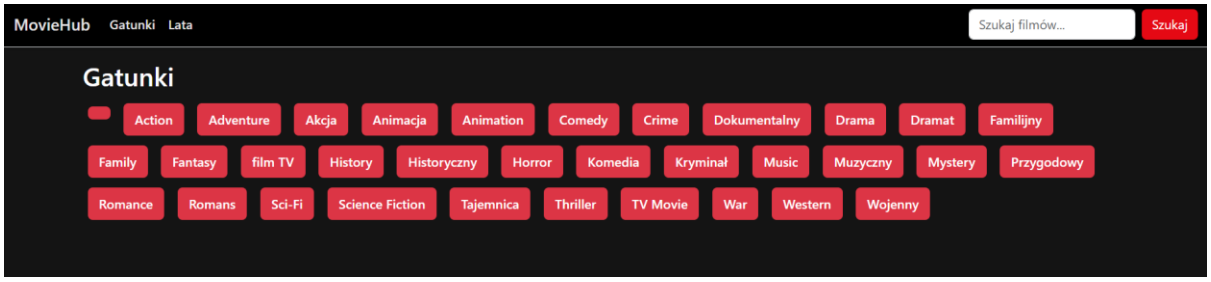
Kliknięcie w kartę przenosi użytkownika do strony szczegółów.



Rysunek 1. "Strona główna aplikacji – lista popularnych filmów"

5.2 Filtrowanie według gatunku

Aplikacja umożliwia przeglądanie filmów według gatunków. Gatunki są prezentowane jako czerwone przyciski. Po wybraniu jednego z nich użytkownik widzi listę filmów należących do danej kategorii.



Rysunek 2. "Widok dostępnych gatunków filmowych w aplikacji"

5.3 Filtrowanie według roku

Użytkownik może również przeglądać filmy według roku produkcji. Lista lat jest generowana automatycznie na podstawie danych w bazie.

Rysunek 3. "Lista filmów w wybranym gatunku"

5.4 Strona szczegółów filmu

Strona szczegółów zawiera:

- duży plakat,
- tytuł,
- gatunek,
- rok produkcji,
- reżysera,
- głównych aktorów,
- opis fabuły (PL lub EN),
- ocenę filmu.

Widok został zaprojektowany tak, aby był czytelny i estetyczny.

Rysunek 4. "Strona szczegółów filmu w aplikacji MovieHub"

5.5 Import danych z TMDB

Aplikacja posiada mechanizm importu, który:

1. pobiera listę popularnych filmów,
2. pobiera szczegóły i obsadę,
3. zapisuje dane do bazy SQLite,
4. sprawdza duplikaty,
5. stosuje fallback językowy.

Dzięki temu baza danych jest zawsze aktualna i kompletna.

6. Najciekawsze fragmenty kodu

W trakcie realizacji projektu powstało wiele elementów kodu, jednak kilka z nich zasługuje na szczególne wyróżnienie ze względu na swoją funkcjonalność, złożoność lub znaczenie dla działania aplikacji. Poniżej przedstawiono najważniejsze fragmenty wraz z krótkim omówieniem.

6.1 Mechanizm fallbacku językowego (PL → EN)

Jednym z kluczowych problemów było to, że TMDB nie zawsze udostępnia polskie opisy filmów. Aby zapewnić spójność danych, zaimplementowano mechanizm automatycznego pobierania opisu angielskiego, jeśli polski nie jest dostępny.

C#
```
if (string.IsNullOrWhiteSpace(overview))
{
    string enUrl =
$"https://api.themoviedb.org/3/movie/{movieId}?api_key={_apiKey}&language=
en-US";
    string enJson = await client.GetStringAsync(enUrl);
    var enDetails = JObject.Parse(enJson);
    overview = enDetails["overview"]?.ToString();
}
```

Ten fragment kodu gwarantuje, że użytkownik zawsze otrzyma pełny opis filmu, niezależnie od dostępności tłumaczeń.

6.2 Filtrowanie filmów według gatunku

Prosta, ale bardzo praktyczna funkcja, która umożliwia szybkie przeglądanie filmów z wybranej kategorii.

C#
```
public IActionResult ByGenre(string genre)
{
    var movies = _context.Movies
        .Where(m => m.Genre.Contains(genre))
        .ToList();

    ViewBag.Genre = genre;
    return View(movies);
}
```

Dzięki temu użytkownik może w kilka sekund znaleźć filmy odpowiadające jego preferencjom.

6.3 Stylizacja przycisków filtrów (CSS)

Interfejs aplikacji został wzbogacony o nowoczesne czerwone przyciski inspirowane stylem Netflixa.

css
```
.filter-button {
    padding: 8px 14px;
    margin: 6px;
    background-color: #dc3545;
    color: white;
    border-radius: 6px;
    text-decoration: none;
    transition: 0.2s;
}
```

Dzięki temu aplikacja wygląda nowocześnie i jest przyjemna w obsłudze.

7. Struktura projektu

Projekt został zorganizowany zgodnie z architekturą ASP.NET Core MVC, co zapewnia przejrzystość, modularność oraz łatwość dalszego rozwoju. Poniżej przedstawiono strukturę katalogów wraz z krótkim opisem ich roli.

7.1 Główne katalogi projektu

```
Connected Services
Dependencies
```

```
Properties
```

Elementy techniczne projektu, generowane automatycznie przez środowisko Visual Studio.

7.2 wwwroot

```
wwwroot/
    css/
    js/
    lib/
    favicon.ico
```

Katalog zawierający pliki statyczne aplikacji, takie jak arkusze stylów, skrypty JavaScript, biblioteki front-endowe oraz ikonę strony.

7.3 Controllers

```
Controllers/
    HomeController.cs
    MoviesController.cs
```

Kontrolery odpowiedzialne za obsługę żądań użytkownika.

- **HomeController** – obsługuje stronę główną i podstawowe widoki.
- **MoviesController** – odpowiada za wyświetlanie filmów, filtrowanie, wyszukiwanie oraz szczegóły.

7.4 Data

```
Data/
    ApplicationDbContext.cs
```

Warstwa dostępu do danych. Zawiera konfigurację bazy SQLite oraz definicję kontekstu Entity Framework Core.

7.5 Migrations

Katalog generowany automatycznie przez EF Core, zawierający migracje bazy danych.

7.6 Models

```
Models/
    ErrorViewModel.cs
    Movie.cs
    Movie/
    TmdbResponse.cs
```

Modele danych używane w aplikacji:

- **Movie.cs** – główny model reprezentujący film.
- **TmdbResponse.cs** – pomocniczy model do obsługi odpowiedzi z API TMDB.
- **ErrorViewModel** – model obsługi błędów.

7.7 Services

```
Services/
    TMDbImporter.cs
```

Warstwa logiki biznesowej odpowiedzialna za integrację z API TMDB oraz import danych do bazy.

7.8 Views

```
Views/
    Home/
        Index.cshtml
        Privacy.cshtml

    Movies/
        ByGenre.cshtml
        ByYear.cshtml
        Create.cshtml
        Delete.cshtml
        Details.cshtml
        Edit.cshtml
        Genre.cshtml
        Index.cshtml
        Search.cshtml
        Years.cshtml

    Shared/
        _Layout.cshtml
```

```
        _ValidationScriptsPartial.cshtml
        Error.cshtml
        _ViewImports.cshtml
        _ViewStart.cshtml
```

Widoki Razor odpowiedzialne za warstwę prezentacji.

- **Home** – widoki strony głównej.
- **Movies** – wszystkie widoki związane z filmami (lista, filtry, szczegóły, CRUD).
- **Shared** – wspólne elementy interfejsu, takie jak layout i importy.

7.9 Pliki konfiguracyjne

```
appsettings.json
movies.db
Program.cs
```

- **appsettings.json** – konfiguracja aplikacji i połączenia z bazą.
- **movies.db** – lokalna baza SQLite.
- **Program.cs** – konfiguracja pipeline aplikacji oraz wywołanie importu danych.

8. Podsumowanie

Realizacja projektu pozwoliła mi stworzyć kompletną aplikację webową, która łączy technologię ASP.NET Core MVC, integrację z TMDB API oraz wsparcie sztucznej inteligencji. W trakcie pracy zdobyłem praktyczne doświadczenie w tworzeniu struktury MVC, pracy z bazą danych SQLite, przetwarzaniu danych JSON oraz projektowaniu interfejsu użytkownika. Zastosowanie AI znacząco przyspieszyło proces programowania i pomogło w analizie błędów, co przełożyło się na bardziej dopracowany efekt końcowy.

Aplikacja jest w pełni funkcjonalna, stabilna i spełnia wszystkie założenia projektu. Jednocześnie została zaprojektowana w sposób umożliwiający jej dalszy rozwój. W przyszłości planuję rozszerzyć ją o dodatkowe funkcjonalności, takie jak wyszukiwarka filmów, system logowania użytkowników, możliwość tworzenia list ulubionych oraz bardziej zaawansowane filtrowanie i rekomendacje. Rozbudowa projektu pozwoli mi pogłębiać umiejętności programistyczne i rozwijać aplikację w kierunku bardziej profesjonalnego rozwiązania.

Repozytorium MovieHub

9. Źródła

1. Dokumentacja ASP.NET Core — Microsoft Learn
2. Dokumentacja Entity Framework Core
3. Dokumentacja TMDB API
4. Materiały edukacyjne Microsoft
5. StackOverflow
6. Wsparcie AI (generowanie kodu, analiza błędów, projektowanie UI)